Exercises

2.1 CWE-918 *Server-Side Request Forgery (SSRF) flaws occur whenever a web application is fetching a remote resource without validating the user-supplied URL. It allows an attacker to coerce the application to send a crafted request to an unexpected destination, even when protected by a firewall, VPN, or another type of network access control list (ACL).*

(a) Using the prepared Docker image, run simple HTTP server with SSRF vulnerability:

```
docker run -dp 5001:5001 --name ssrf-ex1 docker.io/mazurkatarzyna/ssrf-ex1
podman run -dp 5001:5001 --name ssrf-ex1 docker.io/mazurkatarzyna/ssrf-ex1
```

If Docker Hub is not responding, use the backup image:

```
docker run -dp 5001:5001 --name ssrf-fix-ex1 ghcr.io/mazurkatarzynaumcs/ssrf-ex1:latest
podman run -dp 5001:5001 --name ssrf-fix-ex1 ghcr.io/mazurkatarzynaumcs/ssrf-ex1:latest
```

Once launched, the application will be available at http://127.0.0.1:5001.

(b) Knowing that the application is vulnerable to a **SSRF** attack, try to fetch the following URLs:

- `http://httpbin.org/html`
- `http://example.com`
- `https://google.com`

(c) Execute the following command in your terminal to open a shell inside the container:

```
docker run -it -p 5002:5002 --name ssrfixex1 docker.io/mazurkatarzyna/ssrf-fix-ex1
podman run -it -p 5002:5002 --name ssrfixex1 docker.io/mazurkatarzyna/ssrf-fix-ex1
```

(d) Inside the container, open the `app.py` file using the `nano` editor and modify the code to remove the SSRF vulnerability by enforcing an allowlist for URLs:

```
nano app.py
```

Hint: The most effective way to prevent SSRF is to isolate resource access in separate networks, enforce strict "deny by default" firewall rules, and log network flows. At the application layer, always sanitize inputs, enforce allowlists for URLs, disable redirects, and avoid denylists since they can be bypassed.

(e) After updating the code, launch the application and navigate to http://127.0.0.1:5002 to verify that the path traversal vulnerability has been successfully mitigated:

```
python3 app.py
```

Once launched, your fixed application will be available at http://127.0.0.1:5002.

(f) Try to fetch the following URLs again. After fixing the vulnerability fetching those URLs should no longer work:

- `http://httpbin.org/html`
- `http://example.com`
- `https://google.com`

2.2 **CVE-2023-4624**, **CWE-918** ***BookStack*** *is a simple, self-hosted, easy-to-use platform for organising and storing information. In version* `v23.06.2` *it's webhook functionality is vulnerable to* ***SSRF*** *attack, which makes it possible to perform a port scan against the local environment.*

(a) Using the prepared Docker compose file, run **BookStack** in the vulnerable version:

```
docker compose -f docker-compose-cve-2023-4624.yml up # if using Docker
podman-compose -f docker-compose-cve-2023-4624.yml up # if using Podman
```

Once launched, the application will be available at `http://10.133.8.10`. Default credentials are `admin@admin.com/password`.

(b) Create a new webhook. In the webhook endpoint field, enter an internal address such as `http://127.0.0.1:<port>`. Enable **All system events**, set the timeout to 1 second, and save. Then trigger an event (e.g. create or edit a page) and return to the webhook edit screen. Examine the **Webhook Status** section - the error message will reveal whether the port is open or closed. An open port typically returns a connection or protocol error, while a closed port returns *"Failed to connect"*.

(c) Using this technique, scan for open ports on internal `10.133.8.0/24` network. Try common ports. Which ones are open? What services might be running behind them? This is a Server-Side Request Forgery (SSRF) vulnerability - the application makes HTTP requests on your behalf from within the internal network.

(d) What services are running on open ports? You can try connect to open ports by using **ncat**. Use **Hydra** to bruteforce login credentials to the service(s) you found. Were you able to log in to the running service(s)?

(e) After logging in into service(s), look for a flag file (`flag.txt`) on the system.

(f) Run **Burp Intruder** and evaluate whether the webhook endpoint can be exploited to perform a DoS attack. What mitigations should be applied and what HTTP responses are appropriate under overload or abuse conditions?

(g) Click **here** to check out how the authors fixed the vulnerability.

(h) After successfully confirming the existence of the vulnerability, estimate its criticality level using the CVSS **calculator**.

(i) Once you have finished the exploitation, stop the application using the following command:

```
docker compose -f docker-compose-cve-2023-4624.yml down -v # if using Docker
podman-compose -f docker-compose-cve-2023-4624.yml down -v # if using Podman
```

2.3 **CVE-2021-40822**, **CWE-918** *GeoServer is an open-source server for sharing geospatial data, written in Java. In versions through 2.18.5 and 2.19.x through 2.19.2, a critical **SSRF** vulnerability exists in the TestWfsPost servlet. This endpoint is part of the **Demo Requests** page - a built-in tool that lets administrators test OGC web service requests (WFS, WMS, WCS) by sending them from the server side and displaying the response. The servlet takes a URL from the user and issues an HTTP request to it server-side. The URL is validated against the Host header of the incoming request, but by spoofing that header an attacker can redirect the server-side request to any internal target. There is no allow-list, IP validation, or protocol restriction. Because of this, attackers can reach internal network resources, scan ports, or retrieve sensitive data such as cloud-provider metadata endpoints. Version 2.19.3 contains a patch for the issue, and versions 2.24.4 / 2.25.2 remove the servlet entirely.*

(a) Using the prepared Docker compose file, run in the vulnerable version:

```
docker compose -f docker-compose-cve-2021-40822.yml up # if using Docker
podman-compose -f docker-compose-cve-2021-40822.yml up # if using Podman
```

Once launched, the application will be available at http://10.133.7.10:8080/geoserver/web/.

(b) The `TestWfsPost` servlet is a built-in GeoServer page that lets you write an OGC request and send it to GeoServer to test whether it works. Using the http://10.133.7.10:8080/geoserver/TestWfsPost endpoint, send the following payload to the server to get in return an XML document describing the WFS service and its available operations and layers:

```
<?xml version="1.0" encoding="UTF-8"?>
<GetCapabilities
    service="WFS"
    version="1.1.0"
    xmlns="http://www.opengis.net/wfs"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/>
```

Click **Submit**. The server sends the request *to itself* and displays the WFS capabilities document. This is the intended, legitimate behaviour of the servlet.

(c) Reproduce the same request from the command line. First, a plain `GET` that talks to the OWS endpoint directly (no SSRF involved):

```
curl "http://10.133.7.10:8080/geoserver/ows?\
service=wfs&version=1.1.1&request=GetCapabilities"
```

(d) Now send the same request *through* the `TestWfsPost` servlet, just as the UI did (if you are unsure, you can see the request in Burp):

```
curl "http://10.133.7.10:8080/geoserver/TestWfsPost" \
  --data-urlencode "url=http://10.133.7.10:8080/geoserver/ows?\
service=wfs&version=1.1.1&request=GetCapabilities" \
  -d "body=" -d "username=" -d "password="
```

Both should return the same XML capabilities document, confirming that the servlet proxies your request server-side.

(e) Now try pointing the servlet at a different internal host. In the Demo Requests UI, change the URL to an address on the internal network, for example:

```
http://10.133.7.100:8888/
```

The request fails. The servlet validates the target URL against the `Host` header of the incoming HTTP request - since the browser sends `Host: 10.133.7.10:8080`, any URL that does not match that host is rejected.

(f) With `curl` you control every header. The servlet trusts the `Host` header to decide which target is allowed, so by spoofing it you can redirect the server-side request to any destination reachable from the server's network interface - internal IP addresses that are not exposed to the outside (e.g. `10.x.x.x`, `172.16.x.x`, `192.168.x.x`), `localhost` services bound to `127.0.0.1`, cloud metadata endpoints such as `http://169.254.169.254/` (AWS, GCP, Azure), or any other host the server can reach but you normally cannot.

(g) Set the `Host` header to match the *target* address so it passes the validation check. If the target host is reachable from the server, its response is returned to you. You have turned GeoServer into an open proxy.

(h) Use the SSRF to probe the internal network. Try varying the IP address and port to locate other services, for example a HTTP server that is not exposed externally. A non-zero HTTP status (or meaningful HTML) indicates a live service. Identify which host and port responds.

(i) Once you discover an open service, use the SSRF to enumerate sensitive files on it.

(j) What's inside the `flag.txt` file? The flag file confirms full read access to internal resources - but the goal is to show that SSRF impact goes beyond confirming that a host is alive: an attacker can systematically harvest configuration, credentials, and system information from services that were never meant to be reachable.

(k) Click **here** to check out how the authors fixed the vulnerability.

(l) After successfully confirming the existence of the vulnerability, estimate its criticality level using the CVSS **calculator**.

(m) Once you have finished the exploitation, stop the application using the following command:

```
docker compose -f docker-compose-cve-2021-40822.yml down -v # if using Docker
podman-compose -f docker-compose-cve-2021-40822.yml down -v # if using Podman
```

2.4 CVE-2023-6124, CWE-918 ***SuiteCRM*** *is a free, open-source Customer Relationship Management (CRM) software. In versions prior to* `7.14.2`*,* `8.4.2`*,* `7.12.14`*, there exists a vulnerability which allows the attacker to perform a Server-Side Request Forgery (SSRF) attack. SuiteCRM has an Inbound Email Account Test Connection function, and doesn't verify or validate the provided URL/IP Address, allowing it to leak internal service information regardless of the protocol.*

(a) Using the prepared Docker compose file, run **SuiteCRM** in the vulnerable version:

```
docker compose -f docker-compose-cve-2023-6124.yml up # if using Docker
podman-compose -f docker-compose-cve-2023-6124.yml up # if using Podman
```

Once launched, the application will be available at `http://10.133.9.10:8080/`. Default credentials are `user/bitnami`.

(b) Run **BurpSuite**, open its browser and point to `http://10.133.9.10:8080/`. Use default credentials to log in.

(c) In user's profile, go to `Inbound Email Accounts`, and click `New Personal Inbound Email Account`. (Or simply use this link: `http://10.133.9.10:8080/index.php?%20module=InboundEmail&action=EditView&is_personal=1&type=personal`) Fill the information, and click `Test connection`, catch request in BurpSuite.

(d) Find the request responsible for connection testing. Send it to **Repeater**. Once in Repeater, carefully analyze the request to understand its structure and behavior.

(e) In the **Repeater** tab, modify the port number in the request. Observe the HTTP status code, the response content, and the response time.

(f) Knowing that the application is vulnerable to a **SSRF** attack, attempt an internal network port scan to identify which ports are open on the server. Try to scan internal network `10.133.9.0/24` and popular ports.

(g) Click **here** to check out how the authors fixed the vulnerability.

(h) After successfully confirming the existence of the vulnerability, estimate its criticality level using the CVSS **calculator**.

(i) Once you have finished the exploitation, stop the application using the following command:

```
docker compose -f docker-compose-cve-2023-6124.yml down -v # if using Docker
podman-compose -f docker-compose-cve-2023-6124.yml down -v # if using Podman
```

2.5 **CVE-2025-53018**, **CWE-918** *Lychee is a free photo-management tool. Prior to version `6.6.13`, a critical **Server-Side Request Forgery (SSRF)** vulnerability exists in the `/api/v2/Photo::fromUrl` endpoint. This flaw lets an attacker instruct the application's backend to make HTTP requests to any URL they choose. Consequently, internal network resources - such as localhost services or cloud-provider metadata endpoints - become reachable. The endpoint takes a URL from the user and calls it server-side via `fopen()` without any safeguards. There is no IP address validation, nor are there any allow-list, timeout, or size restrictions. Because of this, attackers can point the application at internal targets. Using this flaw, an attacker can perform internal port scans or retrieve sensitive cloud metadata. Version `6.6.13` contains a patch for the issue.*

(a) Using the prepared Docker compose file, run `Lychee` in the vulnerable version:

```
docker compose -f docker-compose-cve-2025-53018.yml up # if using Docker
podman-compose -f docker-compose-cve-2025-53018.yml up # if using Podman
```

Once launched, the application will be available at `http://127.0.0.1:53018`.

(b) Run **BurpSuite**, open its browser and point to `http://127.0.0.1:53018`. Click through the application, capturing requests in Burp. Knowing that there is a vulnerability, find the vulnerable endpoint. Hint: vulnerability exists when adding images.

(c) After finding a vulnerable endpoint, forward the request sent to it to the **Repeater**.

(d) Try changing the URL to the address of a locally running HTTP server: `http://simple_http:4444`. Check the server logs using the command:

```
docker logs simple_http
podman logs simple_http
```

(e) After successfully confirming the existence of the vulnerability, estimate its criticality level using the CVSS **calculator**.

(f) Once you have finished the exploitation, stop the application using the following command:

```
docker compose -f docker-compose-cve-2025-53018.yml down -v # if using Docker
podman-compose -f docker-compose-cve-2025-53018.yml down -v # if using Podman
```

2.6 **CVE-2025-22952**, **CWE-918** *memos is a privacy-first, lightweight note-taking service. Easily capture and share your great thoughts. In version `0.23.0`, `memos` is vulnerable to **SSRF** attack. This vulnerability allows attackers to access to IP addresses and protocols on localhost. Vulnerability exists in `memos.api.v1.MarkdownService.GetLinkMetadata` API endpoint, which can be accessed by **grpcurl** by providing a `link` (parameter) to the resource we want to check.*

(a) Using the prepared Docker image, run simple gRCP server, to see how to interact with gRCP servers using **grpcurl**:

```
docker run -dp 5290:5230 --name grcpserv docker.io/mazurkatarzyna/simple-grcp-server:latest
podman run -dp 5290:5230 --name grcpserv docker.io/mazurkatarzyna/simple-grcp-server:latest
```

If Docker Hub is not responding, use the backup image:

```
docker run -dp 5290:5230 --name grcpserv ghcr.io/mazurkatarzynaumcs/simple-grcp-server:latest
podman run -dp 5290:5230 --name grcpserv ghcr.io/mazurkatarzynaumcs/simple-grcp-server:latest
```

Once launched, the application will be available at `http://127.0.0.1:5290`.

Simple gRCP server supports the following API methods:

- `main.ApiService/time` - returns current time, takes no parameters
- `main.ApiService/date` - returns current date, takes no parameters
- `main.ApiService/hello` - takes one parameter, your name (as `name`) and returns greetings, i.e.: `Hello Katarzyna`

(b) Using **grpcurl** to execute all the above API methods and retrieve their responses. If you don't have **grpcurl** installed locally, you can use its Dockerized version with the command below:

```
docker run --network host docker.io/mazurkatarzyna/grpcurl
podman run --network host docker.io/mazurkatarzyna/grpcurl
```

(c) Using the prepared Docker compose file, run memos in the vulnerable version:

```
docker compose -f docker-compose-cve-2025-22952.yml up
```

Once launched, the application will be available at `http://127.0.0.1:5230`.

(d) Given that the application running at `http://127.0.0.1:5230` exposes the API method `memos.api.v1.MarkdownService.GetLinkMetadata` and is vulnerable to SSRF, demonstrate the vulnerability. Can you scan remote or local addresses?

Hint: `memos.api.v1.MarkdownService.GetLinkMetadata` takes a single, `link` parameter, which returns the metadata from the given link.

(e) Click **here** to check out how the authors fixed the vulnerability.

(f) After successfully confirming the existence of the vulnerability, estimate its criticality level using the CVSS **calculator**.

Links

- https://portswigger.net/web-security/ssrf
- https://portswigger.net/burp/vulnerability-scanner
- https://cve.mitre.org/
- https://www.cvedetails.com/
- https://nvd.nist.gov/
- https://owasp.org/www-community/attacks/Server_Side_Request_Forgery
- https://cheatsheetseries.owasp.org/cheatsheets/SSRF_Prevention_Cheat_Sheet.html
- https://owasp.org/Top10/A10_2021-Server-Side_Request_Forgery_(SSRF)